

YILDIZ TECHNICAL UNIVERSITY

Faculty of Education

Computer Networks & Operating Systems Joint Project

DIJITAL KASIF

A Duolingo-style mobile learning application for middle school students

Team Number	2
Group Members	Cem Gedikoğlu Çağrı Kaçmaz İlknur Aksaray Yusuf Kutlu
Computer Networks Topic	Cybersecurity
Operating Systems Topic	File-System Interface
Instructor	Dr. Kiran
GitHub Repository Link	To be added before submission

Table of Contents

1. Project Overview
 2. Course Topic Selection
 3. Target Audience and Learning Objectives
 4. Educational Design
 5. Mobile Application Structure
 6. Technical Implementation
 7. User Interface and Interaction Design
 8. Testing and Evaluation
 9. Limitations and Future Work
 10. Conclusion
- Appendix A. Running the Project
- Appendix B. Repository and Submission Checklist

1. Project Overview

This report presents the joint project prepared for the Computer Networks and Operating Systems courses. The project is a mobile learning application named Dijital Kasif. It is designed for middle school students and aims to teach selected computing concepts through short explanations, everyday-life analogies, mini missions, and quiz questions.

The application follows a Duolingo-style learning flow, but instead of asking learners to solve exercises immediately, it first introduces the subject through guided content. After the explanation part, students answer multiple-choice questions related to the same topic. This structure supports both conceptual understanding and active recall.

Project Identity

Application name	Dijital Kasif
Team number	2
Target platform	Mobile devices
Development approach	React Native + Expo
Target audience	Middle school students
Main language of application content	Turkish

2. Course Topic Selection

The project required selecting one topic from the Computer Networks course schedule and one topic from the Operating Systems course schedule, then combining them in a single educational mobile application. The selected topics were chosen because they can be explained to middle school learners using concrete examples from daily digital life.

Course	Selected Topic	Course Schedule Relation	Reason for Selection
Computer Networks	Cybersecurity	Week 14 topic: Cybersecurity	Relevant to students' daily internet use, passwords, links, Wi-Fi, and safe behavior online.
Operating Systems	File-System Interface	Week 13 topic: File-System Interface	Easy to adapt with concrete examples such as files, folders, file extensions, and file paths.

The two selected topics were connected under the theme of digital literacy. Cybersecurity teaches students how to protect themselves online, while the file-system topic teaches them how digital information is organized on a device.

3. Target Audience and Learning Objectives

The target group of the application is middle school students. Therefore, the content avoids highly technical terminology and presents each concept with familiar examples such as school bags, locks, folders, libraries, and addresses.

Computer Networks Learning Objectives

- Define cybersecurity in a simple and age-appropriate way.
- Recognize the features of a strong password.
- Explain the risk of phishing links and suspicious messages.
- Identify safer behaviors when using shared or public Wi-Fi networks.

Operating Systems Learning Objectives

- Explain what a file system does in an operating system.
- Distinguish between files and folders.
- Recognize common file extensions such as .jpg, .pdf, and .mp4.
- Understand that a file path shows the location of a file on a device.

4. Educational Design

The instructional model of the application is based on a simple sequence: learn, relate, apply, and check. Each lesson starts with explanation cards. These cards present the main concept, a short daily-life analogy, and a mini mission. After learners complete the explanation cards, they move to the quiz section.

Stage	Purpose	Example in the Application
Learn	Introduce the topic before questions.	Cybersecurity and file-system cards.
Relate	Connect abstract concepts to familiar situations.	Password as a lock; file path as an address.
Apply	Use a mini scenario to encourage decision-making.	Suspicious link scenario; organizing mixed files.
Check	Measure understanding with immediate quiz feedback.	Multiple-choice questions with correct/incorrect responses.

Why a Duolingo-style Design Was Used

The Duolingo-style design was selected because it supports short learning sessions, visible progress, and immediate feedback. However, the application differs from a pure trial-and-error model because it first provides a topic explanation. This makes the design more suitable for middle school students who may be encountering the concepts for the first time.

5. Mobile Application Structure

The application has four main screens: Home, Lesson, Quiz, and Result. The learner starts from the home screen, selects one of the learning missions, reads the lesson cards, answers quiz questions, and finally sees the score and progress result.

Screen	Function	Main Elements
Home Screen	Introduces the app and shows the learning missions.	App title, progress bar, lesson cards.
Lesson Screen	Presents topic explanations before the quiz.	Concept card, analogy box, mini mission.
Quiz Screen	Checks student understanding.	Question, answer options, feedback message.
Result Screen	Shows performance after the quiz.	Score, completion message, return button.

Learning Flow

Home Screen -> Select Lesson -> Read Explanation Cards -> Complete Mini Mission -> Answer Quiz Questions -> View Result -> Return Home

6. Technical Implementation

The project was implemented using React Native and Expo. This technology choice makes it possible to run the application on mobile devices through Expo Go without building a native Android or iOS package during the development stage.

Project File Structure

```
Team2_OS_ComNets/
|-- App.js
|-- app.json
|-- package.json
|-- README.md
|-- .gitignore
|-- index.js
`-- data/
    |-- lessons.js
```

Main Files

File	Role	Explanation
App.js	Main application file	Controls screen transitions, quiz logic, scoring, progress, and interface rendering.
data/lessons.js	Lesson data file	Stores lesson cards, objectives, missions, quiz questions, correct answers, and feedback texts.
package.json	Project configuration	Lists project scripts and dependencies required to run the Expo application.
app.json	Expo configuration	Defines the Expo application name, slug, version, and platform settings.
README.md	Project documentation	Explains the application, selected topics, setup process, and delivery notes.

Quiz and Progress Logic

- Each lesson contains a separate quiz array in the lesson data file.
- Each quiz question has options, a correct answer index, and a feedback message.
- When the learner selects an option, the app immediately checks whether the answer is correct.
- Correct answers increase the score, and the final result is displayed at the end of the lesson.
- Completed lessons are marked on the home screen to show overall progress.

7. User Interface and Interaction Design

The interface was designed to be simple, colorful, and appropriate for middle school students. Each lesson has its own color and icon, which helps students distinguish between the Computer Networks topic and the Operating Systems topic.

- Large titles and short paragraphs were used to reduce cognitive load.
- Progress bars were added to make learning progress visible.
- Cards and rounded buttons were used to create a mobile-friendly learning environment.
- Immediate feedback was included so that students can understand their mistakes quickly.
- The content is written in Turkish because the target learners are middle school students in Turkey.

8. Testing and Evaluation

The project was tested according to the main user flow of the application. The purpose of testing was to check whether the app can start correctly, display both lessons, move between screens, evaluate answers, and show the final result.

Test Case	Expected Result	Status
Start the app with npm start	Expo development server starts.	Prepared
Open the app using Expo Go	Home screen appears on the phone.	Prepared
Select Cybersecurity lesson	Lesson cards are displayed.	Prepared
Select File-System lesson	Lesson cards are displayed.	Prepared
Answer quiz questions	Feedback and score are shown.	Prepared
Finish a lesson	Result screen and progress update are shown.	Prepared

Final testing should be completed after the project folder is downloaded and run on the group computer. The GitHub repository should also show the final code and commit activity before submission.

9. Limitations and Future Work

The current version is a milestone-level educational prototype. It focuses on a clear learning flow and basic interactivity rather than advanced account management or online storage.

Current Limitations

- The application does not include user login or student accounts.
- Progress is stored only during the current app session.
- There is no teacher dashboard or online database.
- The prototype includes two lessons, one for each course topic.

Possible Future Improvements

- Add more lessons from both course schedules.
- Add animations or short interactive simulations for packet transfer and file organization.
- Store student progress locally or in a cloud database.
- Add badges, levels, and streaks to strengthen gamification.
- Add a teacher panel for viewing class progress.

10. Conclusion

Dijital Kasif combines two course topics into one mobile educational application. From Computer Networks, the project teaches Cybersecurity; from Operating Systems, it teaches the File-System Interface. Both topics were adapted for middle school students using simple explanations, relatable examples, mini missions, and quizzes.

The project demonstrates how university-level computing topics can be simplified and transformed into an accessible learning experience for younger students. The mobile structure, progress feedback, and quiz-based practice support an engaging and understandable learning process.

Appendix A. Running the Project

The project can be run on a computer with Node.js installed. After extracting the project folder, the following steps should be completed:

1. Open the Team2_OS_ComNets folder in Visual Studio Code.
2. Open the terminal inside the project folder.
3. Run `npm install` to install dependencies.
4. Run `npm start` to start the Expo development server.
5. Install Expo Go on a mobile phone.
6. Scan the QR code shown in the terminal or browser window.

Commands:

```
npm install
```

```
npm start
```

Appendix B. Repository and Submission Checklist

Before submission, the group should check the following requirements:

- ☐ The GitHub repository has been created.
- ☐ All project files have been pushed to GitHub.
- ☐ All group members have joined the GitHub repository as collaborators.
- ☐ The GitHub repository link has been pasted into this report.
- ☐ The GitHub repository link has been added to the Group Info Excel Sheet.
- ☐ The report includes the names of all group members.
- ☐ The final ZIP file has been renamed according to the course requirement.
- ☐ The final ZIP file size is under 20 MB.
- ☐ Only one group submission is uploaded to online.yildiz.edu.tr.

Repository Link Placeholder

GitHub Repository Link: <https://github.com/cemgedikoglu/DijitalKasif>